

🔗 See Topic 15 → Greedy Masterclass

🎯 **GOAL:** Master Interval Algorithms! Learn the 4 Canonical Patterns (Merge, Insert, Overlap Count, Sweep Line), event timeline sorting, Difference Arrays, and solve all NeetCode Interval questions!

💡 **AHA MOMENT #1: THE UNIVERSAL OVERLAP & MERGE FORMULA**

For two sorted intervals $A = [A_s, A_e]$ and $B = [B_s, B_e]$ where $A_s \leq B_s$:

- Overlap Condition:** They overlap IF AND ONLY IF $B_s \leq A_e$!
- Merged Bounds:** The combined merged interval is $[A_s, \max(A_e, B_e)]$!
- Separate Bounds:** If $B_s > A_e$, they DO NOT overlap, and A can be pushed to result immediately!

1 CORE INTERVAL TERMINOLOGY

- Closed Interval** $[a, b]$: Both endpoints included ($a \leq x \leq b$).
- Open Interval** (a, b) : Endpoints excluded ($a < x < b$).
- Sorting Prerequisite:** Sorting intervals by 'start' time reduces $O(N^2)$ checks to $O(N \log N)$ + single $O(N)$ pass!
- Active End Pointer:** Tracking 'prevEnd' to check overlap with current interval.

2 3 CASES OF TWO SORTED INTERVALS

Case	Visual Layout	Condition	Action
1. Separate	$[1, 3]$ and $[5, 7]$	$B_s > A_e$ ($5 > 3$)	Add A to result; set $A = B$.
2. Partial Overlap	$[1, 5]$ and $[3, 8]$	$B_s \leq A_e$ ($3 \leq 5$)	Merge: $A_e = \max(5, 8) = 8$.
3. Complete Subsumption	$[1, 10]$ and $[3, 5]$	$B_s \leq A_e$ ($3 \leq 10$)	Merge: $A_e = \max(10, 5) = 10$.

🚀 **KEY TAKEAWAY:** Always sort intervals by start time $(a, b) \rightarrow \text{Integer.compare}(a[0], b[0])$ before attempting to merge!

**💡 AHA MOMENT #2: THE 4 INTERVAL PATTERN FAMILIES**

Every interval question in FAANG interviews maps directly to one of 4 patterns:

1. **Merge Pattern:** Combine overlapping intervals in a single sorted pass.
2. **Insert Pattern:** Place new interval into pre-sorted list using 3-stage partition.
3. **Overlap Count (Meeting Rooms II):** Track maximum concurrent active intervals using Min-Heap or Sweep Line.
4. **Sweep Line / Difference Array:** Sort start ``+1`` and end ``-1`` events across time.

1 PATTERNS 1 & 2: MERGE VS INSERT

- **Merge:** Sort array by ``a[0]``. Compare ``inv[0]`` with ``lastMerged[1]``.
- **Insert:** 3-stage scan: (1) Add before ``newStart``, (2) Merge overlap while ``inv[0] <= newEnd``, (3) Add remaining after.

2 PATTERNS 3 & 4: OVERLAP COUNT VS SWEEP LINE

- **Overlap Count:** Min-Heap storing active ``end`` times. Heap size = concurrent rooms!
- **Sweep Line:** Split into ``(start, +1)`` and ``(end, -1)`` events. Sort events by time and track prefix sum!

🚩 **SWEEP LINE TIE-BREAKER RULE:** When a start event ``+1`` and end event ``-1`` occur at the exact same timestamp, process ``-1`` FIRST if bounds are non-inclusive, or ``+1`` FIRST if inclusive!

💡 AHA MOMENT #3: SINGLE PASS WITH LAST-MERGED POINTER

After sorting intervals by `start` time, maintain a running `merged` list.

At each step, compare current interval `inv` with `last = merged.get(merged.size() - 1)`:

- If `inv[0] <= last[1]` → Merge: `last[1] = Math.max(last[1], inv[1])`.
- Else → Push `inv` as a new separate interval!

🔪 MERGE INTERVALS (LC 56) $O(N \log N)$ IMPLEMENTATION

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    List<int[]> merged = new ArrayList<>();
    for (int[] inv : intervals) {
        if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < inv[0]) {
            merged.add(inv); // Non-overlapping
        } else {
            merged.get(merged.size() - 1)[1] = Math.max(merged.get(merged.size() - 1)[1], inv[1]);
        }
    }
    return merged.toArray(new int[merged.size()][2]);
}
```

🔑 **KEY TAKEAWAY:** `merged.get(merged.size() - 1)[1]` acts as the running active end boundary!

💡 **AHA MOMENT #4: THE 3-STAGE LINEAR SCAN (NO RE-SORTING NEEDED!)**

Since input 'intervals' is ALREADY sorted, insert 'newInterval' in $O(N)$ time without re-sorting:

1. **Stage 1 (Before):** Add all intervals ending before 'newInterval.start' ('inv[1] < newInterval[0]').
2. **Stage 2 (Merge Overlap):** While 'inv[0] <= newInterval[1]', expand bounds: 'newInterval[0] = min', 'newInterval[1] = max'. Add merged 'newInterval'.
3. **Stage 3 (After):** Add all remaining intervals after 'newInterval.end'.

🔗 **INSERT INTERVAL (LC 57) $O(N)$ IMPLEMENTATION**

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> res = new ArrayList<>();
    int i = 0, n = intervals.length;
    // Stage 1: Add all before newInterval
    while (i < n && intervals[i][1] < newInterval[0]) res.add(intervals[i++]);
    // Stage 2: Merge overlapping intervals
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.add(newInterval); // Add merged newInterval
    // Stage 3: Add remaining intervals
    while (i < n) res.add(intervals[i++]);
    return res.toArray(new int[res.size()][]);
}
```

🔑 **KEY TAKEAWAY:** 3-stage linear scan solves Insert Interval in $O(N)$ time and $O(N)$ space without sorting!

💡 **AHA MOMENT #5: MIN-HEAP FOR ACTIVE CONCURRENT ROOMS**

In **Meeting Rooms II (LC 253)**, sort meetings by `start` time. Maintain a **Min-Heap** storing active meeting `end` times!

- If `minHeap.peek() <= meeting.start`, the earliest meeting has ENDED! Free the room (`minHeap.poll()`).
- Push current `meeting.end` to heap (`minHeap.add(meeting.end)`).
- At the end, `minHeap.size()` equals the minimum number of meeting rooms required!

🔧 **MEETING ROOMS II (LC 253) MIN-HEAP IMPLEMENTATION**

```
public int minMeetingRooms(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // Stores end times
    for (int[] inv : intervals) {
        if (!minHeap.isEmpty() && minHeap.peek() ≤ inv[0]) {
            minHeap.poll(); // Free room!
        }
        minHeap.add(inv[1]); // Allocate room
    }
    return minHeap.size();
}
```

📌 **KEY TAKEAWAY:** `minHeap.size()` represents the peak concurrent overlap of active intervals!

💡 AHA MOMENT #6: DISCRETE EVENT TIMELINE SWEEPING

Instead of storing full interval ranges `[start, end]`, break each interval into two discrete events:

1. **Start Event:** `(start, +val)` (Capacity increased / room occupied)
2. **End Event:** `(end, -val)` (Capacity decreased / room freed)

Sort events by time. Sweep across time accumulating running sum to track peak usage!

🚗 CAR POOLING (LC 1094) SWEEP LINE / DIFFERENCE ARRAY

```
public boolean carPooling(int[][] trips, int capacity) {
    int[] diff = new int[1001]; // Difference array over timeline
    for (int[] trip : trips) {
        int numPassengers = trip[0], from = trip[1], to = trip[2];
        diff[from] += numPassengers; // Pickup (+val)
        diff[to] -= numPassengers;   // Dropoff (-val)
    }
    int currPassengers = 0;
    for (int p : diff) {
        currPassengers += p;
        if (currPassengers > capacity) return false; // Over capacity!
    }
    return true;
}
```

📌 **KEY TAKEAWAY:** Difference Array `diff[from] += val; diff[to] -= val;` achieves $O(N + T)$ timeline sweeping!

💡 AHA MOMENT #7: TWO POINTERS INTERSECTION & ADVANCE RULE

For two sorted lists `firstList` and `secondList`:

1. **Intersection Bounds:** `start = max(A.start, B.start)`, `end = min(A.end, B.end)`.
2. **Valid Check:** If `start <= end`, add `[start, end]` to result!
3. **Advance Rule:** Advance pointer of the interval that finishes EARLIEST ('if (A.end < B.end) i++; else j++;')!

🔗 INTERVAL LIST INTERSECTIONS (LC 986) IMPLEMENTATION

```
public int[][] intervalIntersection(int[][] firstList, int[][] secondList) {
    List<int[]> res = new ArrayList<>();
    int i = 0, j = 0;
    while (i < firstList.length && j < secondList.length) {
        int start = Math.max(firstList[i][0], secondList[j][0]);
        int end = Math.min(firstList[i][1], secondList[j][1]);
        if (start <= end) res.add(new int[]{start, end}); // Valid intersection!
        if (firstList[i][1] < secondList[j][1]) i++; // Advance earliest end
        else j++;
    }
    return res.toArray(new int[res.size()][]);
}
```

🚩 **KEY TAKEAWAY:** Advancing the pointer with the smaller end time ensures no potential future overlaps are skipped!

🌿 INTERVAL INTERVIEW DECISION TREE (MERMAID)

📄 PRINTABLE INTERVAL CHEAT SHEET

Pattern Type	Target Problem	Algorithm / Strategy	Complexity
Merge Pattern	Merge Intervals (LC 56)	Sort Start Time → `last[1] = max(last[1], inv[1])`	$O(N \log N)$ Time, $O(N)$ Space
Insert Pattern	Insert Interval (LC 57)	3-Stage Scan (Before, Merge Overlap, After)	$O(N)$ Time, $O(N)$ Space
Meeting Rooms II	Meeting Rooms II (LC 253)	Sort Start Time → Min-Heap tracking end times	$O(N \log N)$ Time, $O(N)$ Space
Sweep Line	Car Pooling (LC 1094)	Difference Array `diff[from]+=val; diff[to]-=val`	$O(N + T)$ Time, $O(T)$ Space
Intersections	List Intersections (LC 986)	`max(start, min(end)`; Advance earliest end	$O(M + N)$ Time, $O(1)$ Space

 **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!

1. MERGE INTERVALS TEMPLATE

```
int[][] merge(int[][] invs) {
    Arrays.sort(invs, (a, b) → Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    for (int[] x : invs) {
        if (res.isEmpty() || res.get(res.size() - 1)[1] < x[0])
            res.add(x);
        else res.get(res.size() - 1)[1] = Math.max(res.get(res.size()
- 1)[1], x[1]);
    }
    return res.toArray(new int[res.size()][1]);
}
```

2. INSERT INTERVAL TEMPLATE

```
int[][] insert(int[][] invs, int[] newInv) {
    List<int[]> res = new ArrayList<>(); int i = 0, n = invs.length;
    while (i < n && invs[i][1] < newInv[0]) res.add(invs[i++]);
    while (i < n && invs[i][0] ≤ newInv[1]) {
        newInv[0] = Math.min(newInv[0], invs[i][0]);
        newInv[1] = Math.max(newInv[1], invs[i][1]); i++;
    }
    res.add(newInv);
    while (i < n) res.add(invs[i++]);
    return res.toArray(new int[res.size()][1]);
}
```

 **REUSABLE CODE:** Copy-paste these verified templates directly into your interview whiteboard or IDE!

3. SWEEP LINE DIFFERENCE ARRAY TEMPLATE

```
int maxOverlapSweepLine(int[][] events, int maxTime) {
    int[] diff = new int[maxTime + 1];
    for (int[] e : events) {
        diff[e[0]] += e[2]; // Start event +val
        diff[e[1]] -= e[2]; // End event -val
    }
    int curr = 0, maxOverlap = 0;
    for (int v : diff) {
        curr += v; maxOverlap = Math.max(maxOverlap, curr);
    }
    return maxOverlap;
}
```

4. LIST INTERSECTIONS TEMPLATE

```
int[][] intersections(int[][] A, int[][] B) {
    List<int[]> res = new ArrayList<>(); int i = 0, j = 0;
    while (i < A.length && j < B.length) {
        int s = Math.max(A[i][0], B[j][0]), e = Math.min(A[i][1], B[j]
[1]);
        if (s <= e) res.add(new int[]{s, e});
        if (A[i][1] < B[j][1]) i++; else j++;
    }
    return res.toArray(new int[res.size()][2]);
}
```

5. MINIMUM PLATFORMS REQUIRED (TWO POINTER ARRIVAL / DEPARTURE SWEEP)

```
int findPlatform(int[] arr, int[] dep, int n) {
    Arrays.sort(arr); Arrays.sort(dep);
    int plat = 1, maxPlat = 1, i = 1, j = 0;
    while (i < n && j < n) {
        if (arr[i] <= dep[j]) { plat++; i++; } // Train arrives before previous departs
        else { plat--; j++; } // Train departs
        maxPlat = Math.max(maxPlat, plat);
    }
    return maxPlat;
}
```

🚀 **KEY TAKEAWAY:** Memorize these core templates to solve 95% of FAANG Interval problems!

1. LEETCODE 252 — MEETING ROOMS I

Easy
★★★★★

Sort +
Overlap
Check

AmazonMeta

Problem:
Determine if a person could attend all meetings (no overlapping meetings).

```
public boolean
canAttendMeetings(int[][]
intervals) {
    Arrays.sort(intervals, (a, b)
    → Integer.compare(a[0], b[0]));
    for (int i = 1; i <
intervals.length; i++) {
        if (intervals[i][0] <
intervals[i - 1][1]) return
false; // Overlap!
    }
    return true;
}
```

2. OVERLAP CONDITION MATRIX

Interval A	Interval B	Condition	Overlaps?
'[0, 30]'	'[5, 10]'	'5 < 30'	✗ YES (Conflict)
'[5, 10]'	'[15, 20]'	'15 >= 10'	✓ NO (Safe)
'[0, 5]'	'[5, 10]'	'5 == 5'	✓ NO (Back-to-back safe)

3. FAANG INTERVIEW COMMUNICATION SCRIPT FOR LC 252

 **Candidate Script:** "To check if a person can attend all meetings, any overlap between two meetings creates a scheduling conflict. I will first sort the intervals array by start time in $O(N \log N)$ time. After sorting, adjacent intervals $[i-1]$ and $[i]$ overlap if and only if $i\text{start} < (i-1)\text{end}$. If any such pair exists, return `false`. Otherwise, return `true` in $O(N)$ time and $O(1)$ space."

4. EDGE CASE TRAPS TO HIGHLIGHT

- **Empty or 1-element input:** Return `true` immediately without sorting.
- **Back-to-back meetings:** e.g., `[0, 5]` and `[5, 10]`. Ending at 5 and starting at 5 is NOT an overlap ($5 < 5$ is false).
- **Integer Overflow in Comparator:** Use `Integer.compare(a[0], b[0])` instead of `a[0] - b[0]`.

 **AHA MOMENT (LC 252):** Sorting by start time guarantees that any potential conflict MUST happen between adjacent sorted meetings `intervals[i]` and `intervals[i-1]`!



2. LEETCODE 56 — MERGE INTERVALS

Medium ★★★★★ Single-Pass Accumulation [Amazon](#) [Meta](#)

Problem:

Merge all overlapping intervals into non-overlapping result array.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    for (int[] inv : intervals) {
        if (res.isEmpty() || res.get(res.size() - 1)[1] < inv[0]) res.add(inv);
        else res.get(res.size() - 1)[1] = Math.max(res.get(res.size() - 1)[1], inv[1]);
    }
    return res.toArray(new int[res.size()][]);
}
```

3. LEETCODE 57 — INSERT INTERVAL

Medium ★★★★★ 3-Stage Linear Scan [Amazon](#) [Google](#)

Problem:

Insert 'newInterval' into pre-sorted intervals and merge if necessary.

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> res = new ArrayList<>(); int i = 0, n = intervals.length;
    while (i < n && intervals[i][1] < newInterval[0]) res.add(intervals[i++]);
    while (i < n && intervals[i][0] ≤ newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]); i++;
    }
    res.add(newInterval);
    while (i < n) res.add(intervals[i++]);
    return res.toArray(new int[res.size()][]);
}
```

🔥 **AHA MOMENT (LC 57):** Insert Interval does NOT require sorting! The 3-stage linear scan completes in $O(N)$ time!



4. LEETCODE 253 — MEETING ROOMS II

Medium ★★★★★ Min-Heap Active End Times [Amazon](#) [Meta](#)**Problem:**

Minimum number of meeting rooms required to hold all meetings.

```
public int minMeetingRooms(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int[] inv : intervals) {
        if (!minHeap.isEmpty() && minHeap.peek() ≤ inv[0]) minHeap.poll();
        minHeap.add(inv[1]);
    }
    return minHeap.size();
}
```

5. LEETCODE 435 — NON-OVERLAPPING INTERVALS

Medium ★★★★★ Sort End Time Greedy Removal [Amazon](#)**Problem:**

Minimum number of intervals to remove to make remaining non-overlapping.

```
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[1], b[1]));
    int removals = 0, prevEnd = Integer.MIN_VALUE;
    for (int[] inv : intervals) {
        if (inv[0] ≥ prevEnd) prevEnd = inv[1];
        else removals++;
    }
    return removals;
}
```

💡 **AHA MOMENT (LC 435):** Sorting by END time greedily leaves maximum room for future intervals!



6. LEETCODE 452 — MIN ARROWS TO BURST BALLOONS

Medium ★★★★★ Sort End Time Arrow Shooting Amazon Meta

Problem:

Minimum number of arrows to burst all 2D balloons.

```
public int findMinArrowShots(int[][] points) {
    if (points.length == 0) return 0;
    Arrays.sort(points, (a, b) → Integer.compare(a[1], b[1]));
    int arrows = 1, prevEnd = points[0][1];
    for (int i = 1; i < points.length; i++) {
        if (points[i][0] > prevEnd) {
            arrows++; prevEnd = points[i][1];
        }
    }
    return arrows;
}
```

7. LEETCODE 986 — INTERVAL LIST INTERSECTIONS

Medium ★★★★★ Two Pointers Max Start Min End Amazon Google

Problem:

Find intersection of two pre-sorted lists of closed intervals.

```
public int[][] intervalIntersection(int[][] A, int[][] B) {
    List<int[]> res = new ArrayList<>();
    int i = 0, j = 0;
    while (i < A.length && j < B.length) {
        int start = Math.max(A[i][0], B[j][0]), end = Math.min(A[i][1], B[j][1]);
        if (start ≤ end) res.add(new int[]{start, end});
        if (A[i][1] < B[j][1]) i++; else j++;
    }
    return res.toArray(new int[res.size()][2]);
}
```

★ **KEY TAKEAWAY:** 'Integer.compare(a[1], b[1])' prevents integer subtraction overflow when coordinates are 'Integer.MIN_VALUE' or 'MAX_VALUE'!

**8. LEETCODE 1094 — CAR POOLING****Medium ★★★★★** Sweep Line Difference Array [Amazon](#) [Google](#)**Problem:**

Determine if car can pick up and drop off all passengers without exceeding capacity.

```
public boolean carPooling(int[][] trips, int capacity) {
    int[] diff = new int[1001];
    for (int[] t : trips) {
        diff[t[1]] += t[0]; // Pickup (+val)
        diff[t[2]] -= t[0]; // Dropoff (-val)
    }
    int curr = 0;
    for (int p : diff) {
        curr += p; if (curr > capacity) return false;
    }
    return true;
}
```

9. LEETCODE 759 — EMPLOYEE FREE TIME**Hard ★★★★★** Sorted Merge Gap Extraction [Amazon](#) [Meta](#)**Problem:**

Find common free time intervals for all employees.

```
public List<Interval> employeeFreeTime(List<List<Interval>> schedule) {
    List<Interval> all = new ArrayList<>();
    for (List<Interval> list : schedule) all.addAll(list);
    all.sort((a, b) -> Integer.compare(a.start, b.start));
    List<Interval> res = new ArrayList<>();
    int tempEnd = all.get(0).end;
    for (Interval inv : all) {
        if (inv.start > tempEnd) {
            res.add(new Interval(tempEnd, inv.start)); // Gap found!
            tempEnd = Math.max(tempEnd, inv.end);
        }
    }
    return res;
}
```

FAANG COACH TIP: Employee Free Time finds gaps between merged working intervals ('inv.start > tempEnd')!

★ THE 4 GOLDEN LAWS OF INTERVALS

1. **Overlap Condition:** Two sorted intervals \$A\$ and \$B\$ overlap IF AND ONLY IF `B.start ≤ A.end`.
2. **Sorting Strategy:** Merge & Overlap Count sort by START time; Non-overlapping removal sorts by END time.
3. **3-Stage Insert:** Insert Interval uses 3-stage linear scan (Before, Merge Overlap, After) in $O(N)$ time.
4. **Comparator Safety:** Always use `Integer.compare(a[0], b[0])` to prevent integer overflow subtraction bugs!

🚫 COMMON INTERVAL BUGS & PITFALLS

- **Integer Subtraction Overflow:** Using `'a, b) → a[0] - b[0]'` breaks when coordinates are `'Integer.MIN_VALUE'` or `'MAX_VALUE'`! Use `'Integer.compare(a[0], b[0])'`.
- **Forgetting Array to List Conversion:** Remember `'list.toArray(new int[res.size()][])'`.

📖 NEETCODE & BLIND 75 CONFIDENCE TRACKER

🟢 Easy LC 252 Meeting Rooms I	🟡 Med LC 56 Merge Intervals	🟡 Med LC 57 Insert Interval
🟡 Med LC 253 Meeting Rooms II	🟡 Med LC 435 Non-overlapping	🟡 Med LC 452 Min Arrows
🟡 Med LC 986 Intersections	🟡 Med LC 1094 Car Pooling	🔴 Hard LC 759 Employee Free Time

👉 Next Master Topic: Topic 17 → Bit Manipulation Masterclass

🚩 TOPIC 16 INTERVALS MASTERCLASS COMPLETE! Turn the page to Topic 17: Bit Manipulation Masterclass!